Annotation Analytics

Build it, measure it, explain it

Full Stack Technical Assessment

CVAT 8 hours

ISSUED **6 October 2026 — 8 hours to submit**

Field	Details
Project Name	**Annotation Analytics Assessment**
Client	**Internal**
Developed by	**xis.ai**
Prepared by	Syed Shuja Hussain Shah
Role	Full Stack Discipline Lead, xis.ai
Start Date	6 October 2026
End Date	8 hours from start
Status	Active

Read this document before you start. It takes ten minutes and saves you more.

Contents

1 The Task . . . **2**

2 Time . . . **2**

3 What You Deliver . . . **2**

4 Setup . . . **3**
- 4.1 Sample Data . . . 3

5 Rules . . . **3**
- 5.1 Git . . . 3
- 5.2 What You Submit . . . 4
- 5.3 The Plan . . . 4
- 5.4 Definition of Done . . . 4

6 Objectives . . . **5**

7 The Recording . . . **5**
- 7.1 What To Record . . . 5
- 7.2 The Four Questions . . . 5

8 How You Are Assessed . . . **6**

9 Automatic Rejection . . . **6**

10 Submission . . . **7**

1 The Task

CVAT is an open source tool for annotating images and video. People draw labelled boxes on images in it. Those labels are the product.

CVAT does not show you how many annotations you have of each class. You will add that.

- An API that counts annotations by class.
- A page that shows those counts as a graph.
- Whatever speed and reliability you can prove in the time you have.

> You will work inside a large codebase you did not write. That is the point. How you find your way around someone else's system matters more to us than how fast you type.

2 Time

8 hours of work. Submit within 8 hours of starting.

Docker images and the dataset import run on their own. Start them first. That waiting time does not count.

> **You are not expected to finish the whole list in Section 3.** Very few people do. Stop where you stop, then write down what you did not reach and why.
>
> An honest, unfinished submission scores better than a rushed, broken one. We have never turned anyone down for stopping in the right place.

3 What You Deliver

Work down this list in order.

#	Requirement
1	An API endpoint that returns the number of annotations per class for a task, read from the database.
2	A page in the web interface that calls it.
3	The counts shown as a graph.
4	The page handles two cases cleanly: no data, and a failed request.
5	The endpoint uses CVAT's existing login. Refuse a request with no login, and refuse a user who has no access to that task. Show both working.
6	One speed target for your endpoint. You set it, you measure it, you report the result. Section 6.
7	One filter or grouping beyond the plain count. Your choice. Say why you chose it.
8	The graph updates live as annotations change, over WebSocket.
9	The page recovers when the connection drops and comes back.
10	A decision record, written inside your Plan: the approach you took, the approach you rejected, and what rejecting it cost.

> **Items 1 to 4 are the floor.** Nothing further is assessed until those four work. Do not start item 8 with item 4 unfinished.

4 Setup

Short on purpose. Getting the stack running is not what we are assessing.

```
# Fork https://github.com/cvat-ai/cvat on GitHub, then:
git clone https://github.com/<your-account>/cvat.git
cd cvat
git checkout -b dev-test01
git rev-parse HEAD      # record this, it goes in your docs

export CVAT_HOST=localhost
docker compose up -d
docker exec -it cvat_server bash -ic 'python3 ~/manage.py createsuperuser'
```

Open `http://localhost:8080`. You need Docker with the Compose plugin, about 8 GB of free RAM and 15 GB of free disk.

Put your backend work in a new Django app named `test`.

4.1 Sample Data

Use the public COCO 2017 validation set, so your numbers mean the same as everyone else's.

```
wget http://images.cocodataset.org/zips/val2017.zip
wget http://images.cocodataset.org/annotations/annotations_trainval2017.zip
unzip -q val2017.zip && unzip -q annotations_trainval2017.zip
```

Create a task and upload the images. Then open **Actions**, choose **Upload annotations**, pick format **COCO 1.0** and the file `annotations/instances_val2017.json`.

Use as many images as your machine handles comfortably. Write down how many.

> **You are ready** when you can open a job and see labelled boxes on an image.

5 Rules

These apply no matter how far you get.

5.1 Git

- Work on branch `dev-test01`.
- Commit as you go, in steps that show the work progressing.
- Commit messages say what changed and why. Not `fix` or `wip`.
- **Commit your plan first, before any code.** Git records the order and we read it.

One large commit at the end is an automatic rejection. So is a first commit that already holds the finished feature.

5.2 What You Submit

Three documents, under `docs/` on your branch. **A missing document is an automatic rejection, whatever your code does.**

Document	What goes in it
Plan	How you intend to reach the task. Committed before any code. Section 5.3.
Objectives	The targets you set yourself, and how you measured them. Section 6.
Definition of Done	How you decided the work was finished, ticked with evidence. Section 5.4.

Three documents, not ten. We would rather read three that are thought through than ten that were filled in to tick a box.

5.3 The Plan

One page, committed first. What you will do, in what order, and roughly how long each part should take out of your 8 hours. Include what you have already decided to skip.

If you reach item 10, close the Plan with a short decision record: the approach you took, the approach you rejected, and what rejecting it cost.

A plan that matches what happened is good. A plan that changed, with a note saying why, is better. A plan written at the end to match the result is what we are looking for, and we find it.

5.4 Definition of Done

Your own checklist, written before you start, ticked at the end with a number or a link beside each line. It is how you decide the work is finished, rather than guessing.

Something like this, but yours:

☐ Endpoint returns correct counts, checked against a known task.

☐ Page renders the graph, with the empty and error cases covered.

☐ Objective measured, 5 runs, raw output saved.

☐ Target met, or missed with the reason written down.

☐ Everything I did not finish is listed.

A checklist with every box ticked and no evidence beside it counts as unticked.

6 Objectives

An objective is measurable when two people, running your method on the same machine, would agree on whether you hit it. "The page should feel fast" is not measurable. Nobody can check it.

Write at least one, in this format. The example uses a different problem, so you cannot copy it.

ID	Field	Entry
MO-1	What is measured	Time from opening the task list page to the first row appearing on screen.
	How	Browser performance panel, timed between a mark in the route handler and a mark in the row component.
	Target	Median of 5 runs at or below **300 ms**.
	Conditions	Chrome, cache disabled, local Docker stack, the sample data, nothing else running.
	Not included	Video tasks, and the first load after a cold start.

You pick the target number, then you justify it. A target you clear without doing anything tells us one thing about you. A target you miss honestly tells us something better.

Rules for any number you report:

- It comes from your machine, pasted as raw output.
- Your docs state your CPU, RAM, operating system and the CVAT commit SHA you cloned.
- Measure 5 times. Report the median and the spread.

One number on its own is not a measurement.

7 The Recording

This decides whether your submission is assessed at all.

Use any tool you like while building, including AI assistants. We do not ask and we do not mind. What we check is whether you understand what you handed in, and we check it by watching you explain it.

7.1 What To Record

One Loom, **5 minutes or less**. Roughly:

Time	What you show
1 minute	The feature running live on your screen.
1.5 minutes	Your own code, following one request from the page to the database and back.
2.5 minutes	The four questions below, answered out loud.

7.2 The Four Questions

Answer all four on camera, without reading from notes.

ID	Question
K1	Which file and line computes the per-class count, and why there rather than somewhere else?
K2	Where does CVAT store the label of a drawn shape? Name the model and the field, and say how you got from a task to a label name.
K3	Name one thing in your own code you would reverse, and what changing it would cost.
K4	Name one thing you did not understand, and what you did about it.

How this is checked. We watch the recording with your code open. If an answer describes something your repository does not contain, the recording fails and nothing else is assessed, however good the code is.

"I do not know, here is what I tried" never fails. A confident wrong answer always does.

8 How You Are Assessed

Three areas, in this order of importance.

Area	What we look at
What works	The parts that actually run, shown by evidence in your Definition of Done. You get partial credit for incomplete work you declare. You get nothing for incomplete work you hide.
The three documents	Your Plan, your Objectives and your Definition of Done. Read closely, because there are only three. We look for thinking, not length.
Code presentation	Where your code sits, how it is named, how your commits read, and how contained the change is. No dead code, no commented-out blocks, no stray files.

Your three documents carry more weight than the code itself. The recording gates everything. A perfect feature you cannot explain scores nothing.

9 Automatic Rejection

Any one of these ends the assessment.

☐ Any of the three documents missing.

☐ One large commit, or a first commit holding finished code.

☐ Recording failed.

☐ No recording, or a recording over 5 minutes.

☐ Numbers that cannot have come from the submitted work.

☐ A pull request opened against the real CVAT repository.

10 Submission

- Public fork of CVAT on your own GitHub account, branch `dev-test01`.
- All documentation under `docs/` on that branch.
- One pull request from `dev-test01` into the main branch **of your own fork**.
- Reply to the message that sent you this document with the pull request link and the Loom link, within 8 hours of starting.

> **Do not open a pull request against the real CVAT repository.** It is a live open source project and this is practice work.